PRD - Smart Tourist Safety Portal

Product Requirements Document
Smart Tourist Safety Monitoring and Incident Response System
1. Overview
2. Problem Statement
3. Goals
4. User Roles
5. Functional Requirements
6. Data Model (high-level entities)
7. System Flows
8. Technology Stack
9. Interface Workflow
10. Non-Functional Requirements
11. Suggested Build Phases
12. Success Criteria (for the working portal)
13. Open Questions / Decisions Needed

Product Requirements Document

Smart Tourist Safety Monitoring and Incident Response System

Version: 1.0 **Status:** Draft for Development **Prepared for:** CEP Project — Web Portal Implementation

---

1. Overview

The Smart Tourist Safety Monitoring and Incident Response System is a web portal that improves tourist safety through real-time GPS tracking, geo-fencing, AI-based risk classification, blockchain-backed digital identity, and one-touch SOS emergency reporting. The portal serves two user types — **Tourists** and **Authorities/Responders** — connected through a shared incident and location data layer.

This document defines what needs to be built for a working, demoable portal (web-based), covering scope, features, data model, flows, and technical requirements.

---

2. Problem Statement

Tourists in unfamiliar or remote areas face risks — getting lost, entering restricted zones, accidents, theft, medical emergencies — with no automated way to detect risk or coordinate a fast response. Existing systems rely on manual reporting and phone calls, with no continuous monitoring, no verified digital identity, and no real-time risk classification.

---

3. Goals

- Track tourist location in real time with consent-based privacy controls.

- Detect and alert when a tourist enters a restricted or high-risk zone.
- Classify incidents by AI as Low / Medium / High risk.
- Issue a secure, blockchain-backed Digital Tourist ID with QR verification.
- Let a tourist raise an SOS in one tap, instantly notifying authorities with location and details.
- Give authorities a single dashboard to view tourists, zones, and incidents, and dispatch response.

Non-Goals (out of scope for this build)

- Native mobile apps (Android/iOS) — portal is web-only for now.
- Wearable/IoT device integration.
- Multilingual support and offline mode.
- Production-grade blockchain deployment (a testnet/simulated version is acceptable).

---

4. User Roles

Role	Description
Tourist	Registers, receives Digital ID, is tracked, can raise SOS
Authority / Responder	Monitors dashboard, receives alerts, verifies ID, dispatches response
Admin	Configures geo-fence zones, manages users, views system-wide data

---

5. Functional Requirements

5.1 Tourist Registration & Digital ID

- Tourist signs up with basic details (name, contact, ID proof).
- System generates a unique **Digital Tourist ID** with a QR code.
- ID is recorded on a blockchain (Polygon testnet or simulated ledger) for tamper-proof verification.
- Tourist can view/download their Digital ID.

5.2 GPS Location Tracking

- Tourist’s browser/device shares live location (with explicit consent toggle).
- Location updates are stored and shown on a map (tourist side: own location; authority side: all active tourists).
- Location sharing can be paused/stopped by the tourist at any time.

5.3 Geo-Fencing

- Admin can define zones on a map: **Safe**, **Restricted**, **High-Risk** (drawn as polygons/circles with lat-long boundaries).
- System checks tourist location against zones in real time.
- Entry into a Restricted or High-Risk zone triggers an automatic alert to the tourist and to the authority dashboard.

5.4 AI Risk Classification

- Every incident (geo-fence breach or SOS) is passed through a risk model.
- Model classifies the incident as **Low / Medium / High** based on factors such as zone type, time of day, and incident type.
- Classification is visible on the authority dashboard, sortable/filterable by risk level.

5.5 SOS Emergency Reporting

- One-touch SOS button on the tourist interface.
- On press, system instantly creates an incident with: Tourist ID, current location, timestamp, and

incident type.
- Authority dashboard receives a real-time alert (push/notification-style update).

5.6 Authority Dashboard

- Login for authorities/responders.
- Map view showing all tourist locations and active incidents.
- Incident list/table with filters (risk level, zone, status: open/resolved).
- Ability to verify a tourist's Digital ID (via QR scan or ID lookup).
- Ability to mark an incident as acknowledged/resolved.

5.7 Notifications

- In-app alert banners for geo-fence breaches and SOS events (real-time or near-real-time refresh).

---

6. Data Model (high-level entities)

Entity	Key Fields
Tourist	id, name, contact, digital_id, qr_code, blockchain_hash, created_at
Location	tourist_id, latitude, longitude, timestamp
Zone	id, name, type (safe/restricted/high-risk), boundary_coordinates
Incident	id, tourist_id, type (geo-fence / SOS), location, risk_level, status, timestamp
Authority	id, name, role, assigned_zone(optional)

---

7. System Flows

Tourist Flow

1. Register → receive Digital Tourist ID (QR, blockchain-verified).
2. Enable location sharing → GPS tracked continuously.
3. System checks location against geo-fence zones.
4. On zone breach or SOS press → incident created and sent to authorities.

Authority Flow

1. Log in to dashboard.
2. View live map of tourists, zones, and incidents.
3. Receive AI-classified alerts (Low/Medium/High).
4. Verify tourist's Digital ID.
5. Acknowledge/dispatch response and resolve incident.

---

8. Technology Stack

Layer	Technology	Purpose
Frontend	React (or HTML/CSS/JS) + Tailwind/Bootstrap	Tourist app UI + Authority dashboard UI
Frontend — Maps	Google Maps JavaScript API + Geolocation API	Live map, zone drawing, marker tracking
Backend / API	Node.js (Express) or Python (FastAPI/Flask)	REST API for auth, location, incidents, zones
Realtime layer	WebSocket (Socket.IO) or polling	Push live location + incident updates to dashboard
AI/ML	Python + TensorFlow / scikit-learn, or a rule-based scoring function for MVP	Classify incidents as Low/Medium/High risk

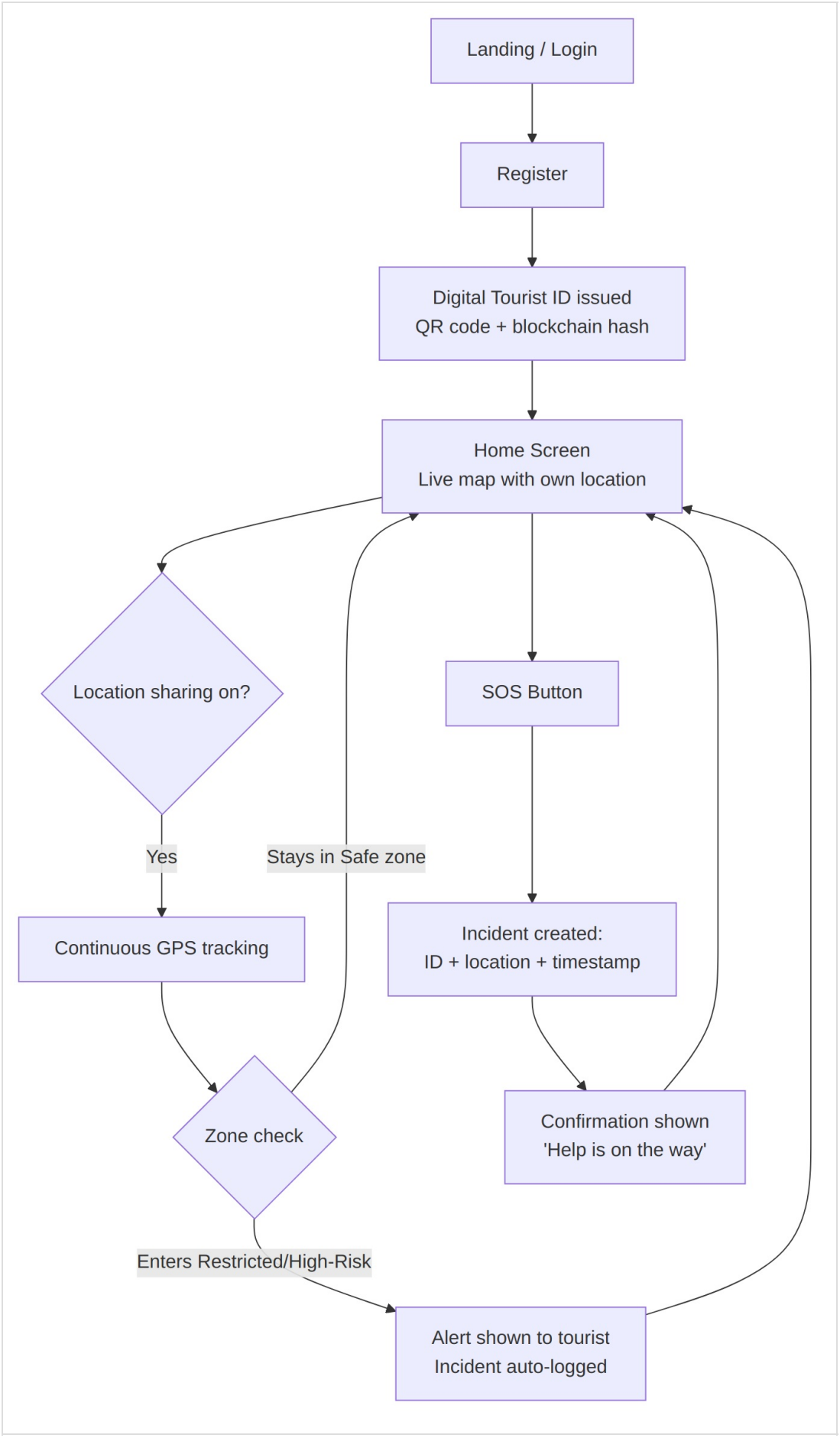
Database	MySQL	Store tourists, locations, zones, incidents, authorities
Blockchain & Identity	Polygon testnet (via Web3.js/Ethers.js) + `qrcode` library	Digital Tourist ID hash + QR generation/verification
Auth	JWT-based session auth	Separate login for Tourist and Authority roles
Hosting (demo)	Local server / Render / Vercel + a managed MySQL instance	Deployable working demo

Suggested stack for the fastest working build: React frontend, Node.js + Express backend, MySQL, Google Maps API, a rule-based risk scorer (swap in TensorFlow later), and a simulated blockchain hash (swap in real Polygon calls once the core flow works end-to-end). This lets every feature be demoed without blocking on blockchain/AI infra first.

---

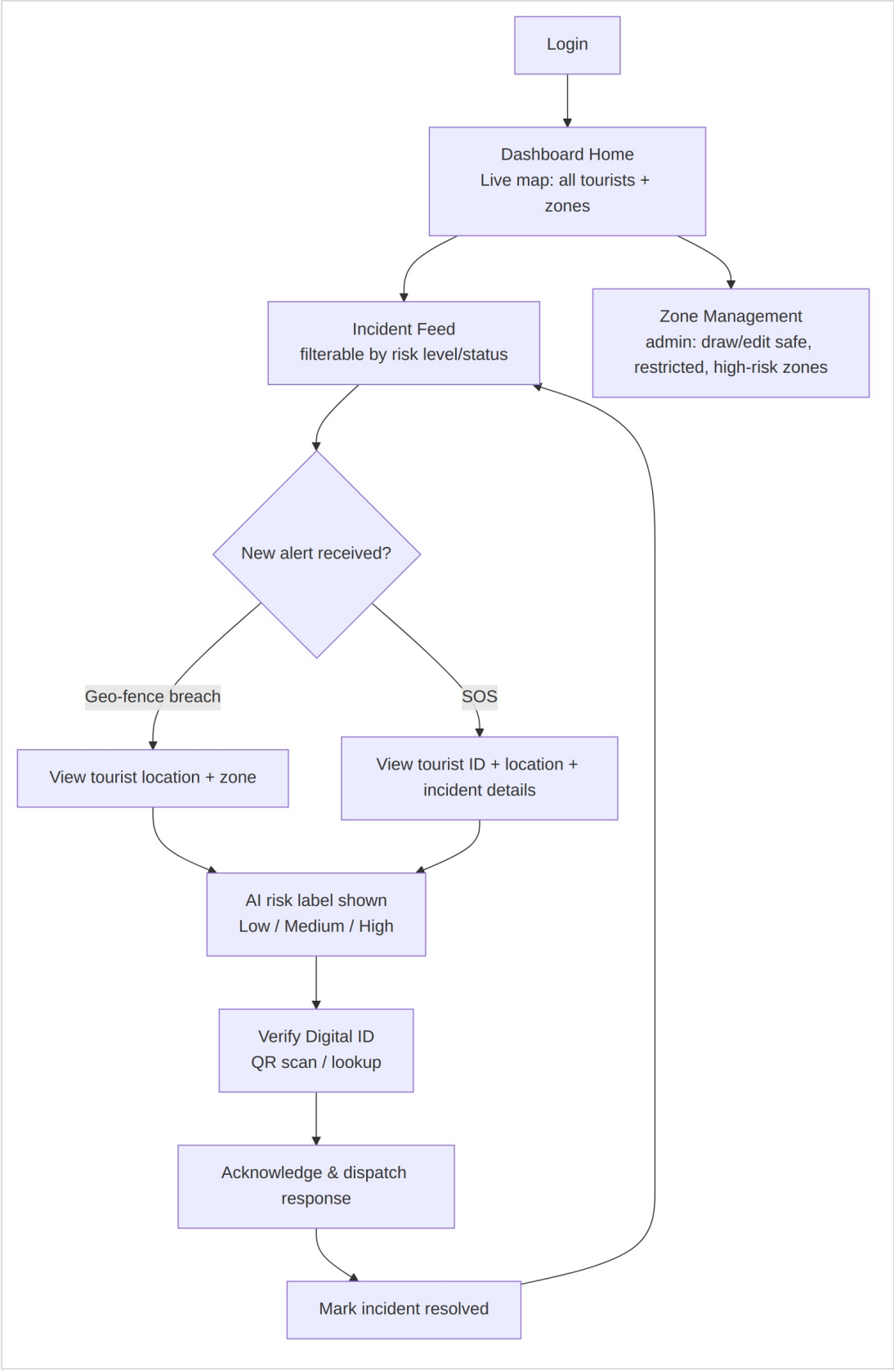
9. Interface Workflow

9.1 Tourist App — Screen Flow



Workflow Diagram 1

9.2 Authority Dashboard — Screen Flow



Workflow Diagram 2

9.3 End-to-End Interaction Summary

Step	Tourist side	Authority side
1	Registers, gets Digital ID	—

2	Enables location sharing	Sees tourist appear on map
3	Moves; system checks geo-fence	—
4	Enters risky zone / presses SOS	Receives real-time alert with AI risk label
5	Sees confirmation on screen	Verifies Digital ID, dispatches help
6	—	Marks incident resolved

---

10. Non-Functional Requirements

- **Real-time-ish updates:** location and incident data should refresh within a few seconds (polling or WebSocket-based).
- **Privacy:** location sharing is opt-in and revocable; Digital ID exposes only verification data, not full personal details, to responders by default.
- **Usability:** SOS action must be reachable in one tap/click from any screen while logged in as a tourist.
- **Reliability:** incident creation must succeed even under poor network conditions (basic retry/queue on the client).

---

11. Suggested Build Phases

Phase	Deliverable
1	Tourist & authority auth, basic UI shell
2	Digital ID generation + QR + blockchain hash (testnet/simulated)
3	GPS tracking + map integration (tourist & authority views)
4	Geo-fence zone creation (admin) + breach detection
5	AI risk classification for incidents
6	SOS button + real-time incident feed on dashboard
7	Polish, QA, demo data, deployment

---

12. Success Criteria (for the working portal)

- A tourist can register, get a Digital ID, and see their live location on a map.
- Entering a defined restricted/high-risk zone triggers a visible alert on both sides.
- Pressing SOS creates an incident visible on the authority dashboard within seconds, with a risk classification attached.
- An authority can verify a Digital ID and mark an incident resolved.

---

13. Open Questions / Decisions Needed

- Real blockchain (Polygon testnet, needs wallet/gas setup) vs. simulated hash-based “blockchain-style” ledger for demo purposes?
- Real AI model (needs training data) vs. rule-based risk scoring for the first working version?
- Hosting/deployment target for the portal (local demo vs. cloud-hosted)?